andreaskoepf and facebook-github-bot Migrate nll_loss from TH to ATen (CPU) (#… 2466dc8 · 6 years ago

Name	Last commit message	Last commit date
..		
cpu	CPU-strided-complex support for compa…	6 years ago
cuda	Migrate sinh and sinh_ from the TH t…	6 years ago
cudnn	Use grad_out for cudnn CTC loss (#27039)	6 years ago
miopen	Allow batch size of 0 in Conv	6 years ago
mkl	Deprecate tensor.data<T>(), and codemo…	6 years ago
mkldnn	Add memory format argument to the cl…	6 years ago
quantized	Add Conv3dInt8 (#28768)	6 years ago
sparse	provide memory format as Contiguous e…	6 years ago
utils	Add aten mkldnn conv2d operator	6 years ago
Activation.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
Activation.h	hardshrink_cpu and hardshrink_backwar…	6 years ago
AdaptiveAveragePooling.cpp	(#24396)	6 years ago
AdaptiveAveragePooling3d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
	Deprecate tensor.data<T>(), and codemo…	6 years ago

AdaptiveMaxPooling2d.cpp		
AdaptiveMaxPooling3d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
AffineGridGenerator.cpp	Add align_corners option to grid_sample…	6 years ago
AveragePool2d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
AveragePool3d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
BatchLinearAlgebra.cpp	Migrate implementations of triu and tril t…	6 years ago
BinaryOps.cpp	CPU-strided-complex support for compa…	6 years ago
BinaryOps.h	Migrate `smooth_l1_loss` from the T…	6 years ago
Col2Im.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
ConstantPadNd.cpp	cleanup warnings	6 years ago
Convolution.cpp	added a check for zero stride	6 years ago
ConvolutionTBC.cpp	Replace AT_CHECK with TORCH_CH…	6 years ago
Copy.cpp	Fix CUDA named tensor `copy_` (#26829)	6 years ago
Copy.h	Revert D18170996: Move type casting t…	6 years ago
Cross.cpp	Replace AT_CHECK with TORCH_CH…	6 years ago
Cross.h	torch.cross' dim default changed to c10::…	6 years ago
DilatedConvolutionUtils.h	Rename previously THNN conv kernels …	6 years ago
DilatedMaxPool2d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
DilatedMaxPool3d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
DispatchStub.cpp	Do not call cpuinfo_initialize() on other t…	6 years ago
DispatchStub.h	Fixing dispatch error in windows debug …	6 years ago
Distance.cpp	cdist performance improvement for eucli…	6 years ago
Distance.h	Backward function for torch.cdist	6 years ago
Distributions.cpp	Make named tensor implementations mo…	6 years ago
Distributions.h	Refactor Random Number Generators in …	6 years ago
Dropout.cpp	Make named tensor implementations mo…	6 years ago

Embedding.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
EmbeddingBag.cpp	Upgrade the deprecated data to data_ptr …	6 years ago
Fill.cpp	Kill _th_zero binding, just use a simple n…	6 years ago
Fill.h	Move fill and fill_diagonal to Fill.cpp, Fi…	6 years ago
FractionalMaxPool2d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
FractionalMaxPool3d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
GridSampler.cpp	Add align_corners option to grid_sample…	6 years ago
GridSampler.h	Add align_corners option to grid_sample…	6 years ago
Im2Col.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
Indexing.cpp	Remove zero_dim_tensors only from _th…	6 years ago
Indexing.h	Improve performance of advanced indexi…	6 years ago
IndexingUtils.h	cleanup warnings	6 years ago
Integration.cpp	numerical integration "trapz" function. (#…	6 years ago
Itertools.cpp	Changed tensor comparison return type f…	6 years ago
LegacyBridge.cpp	Implement multiple dispatch (#26468) (#…	6 years ago
LegacyDefinitions.cpp	Remove zero_dim_tensors only from _th…	6 years ago
LegacyNNDefinitions.cpp	porting convtranspose3d to ATen (#22019)	6 years ago
Lerp.cpp	Rewrite lerp operator to use TensorIterat…	6 years ago
Lerp.h	Rewrite lerp operator to use TensorIterat…	6 years ago
Linear.cpp	Eliminate some unnecessary tensor refco…	6 years ago
LinearAlgebra.cpp	CPU-Strided-Complex Support for reduc…	6 years ago
LinearAlgebraUtils.h	Migrate implementations of triu and tril t…	6 years ago
Loss.cpp	Migrate `smooth_l1_loss` from the T…	6 years ago
LossCTC.cpp	Use grad_out for cudnn CTC loss (#27039)	6 years ago
LossMultiLabelMargin.cpp	Port of multilabel_margin_loss from TH …	6 years ago

LossMultiMargin.cpp	Port of multi_margin_loss from TH to A…	6 years ago
LossNLL.cpp	Migrate nll_loss from TH to ATen (CPU)…	6 years ago
Math.h	Vectorize unary operator erfinv (#26629)	6 years ago
MaxUnpooling.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
Memory.cpp	pin_memory should not copy on already_…	6 years ago
NNPACK.cpp	Modify PyTorch's integration of NNPAC…	6 years ago
NaiveConvolutionTranspose2d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
NaiveConvolutionTranspose3d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
NaiveDilatedConvolution.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
NamedTensor.cpp	Allow `align_to` to take in partially na…	6 years ago
Normalization.cpp	Remove CPU_tensor_apply* from Norm…	6 years ago
Onehot.cpp	Replace AT_CHECK with TORCH_CH…	6 years ago
PackedSequence.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
PixelShuffle.cpp	Canonicalize all includes in PyTorch. (#1…	7 years ago
PointwiseOps.cpp	Turn on BUILD_NAMEDTENSOR per…	6 years ago
PointwiseOps.h	Migrate `smooth_l1_loss` from the T…	6 years ago
Pool.h	Use integer floor division for pooling sha…	6 years ago
Pooling.cpp	Support ceil_mode in quantized maxpool	6 years ago
Pow.cpp	Allocate empty tensor instead of empty_l…	6 years ago
Pow.h	Port `pow` operator from the TH code to …	6 years ago
QuantizedLinear.cpp	Remove fbgemm_is_cpu_supported in fa…	6 years ago
README.md	Add documentation for overload names (…	6 years ago
RNN.cpp	enable double backward for non-cudnn L…	6 years ago
RNN.h	Revert D16238204: Revert D16224780: …	6 years ago
RangeFactories.cpp	CPU-strided-complex support for compa…	6 years ago

ReduceOps.cpp	CPU-Strided-Complex Support for reduc…	6 years ago
ReduceOps.h	Rewrite argmax and argmin as TensorIte…	6 years ago
ReduceOpsUtils.h	cleanup warnings	6 years ago
ReflectionPad.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
Repeat.cpp	Replace AT_CHECK with TORCH_CH…	6 years ago
Repeat.h	Deprecate tensor.data<T>(), and codemo…	6 years ago
ReplicationPadding.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
Resize.cpp	Make resize_as_ generic, so XLA works.…	6 years ago
Resize.h	Zero slice bug (#20914)	6 years ago
ResizeCommon.h	Implement resize_, resize_as_ for named…	6 years ago
Scalar.cpp	Cpu-strided-complex support for binary-…	6 years ago
SharedReduceOps.h	CPU-Strided-Complex Support for reduc…	6 years ago
SobolEngineOps.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
SobolEngineOpsUtils.h	Introduce SobolEngine (#10505)	6 years ago
SoftMax.cpp	Make named tensor implementations mo…	6 years ago
Sorting.cpp	Named tensor support for: index_fill_, in…	6 years ago
Sorting.h	Reapply: optimize topk on cpu using par…	6 years ago
SortingUtils.h	quantized torch.topk (#26486)	6 years ago
SpectralOps.cpp	Replace AT_CHECK with TORCH_CH…	6 years ago
SpectralOpsUtils.h	More use of AT_CHECK and AT_ERRO…	7 years ago
SummaryOps.cpp	Delete toType(const DeprecatedTypePro…	6 years ago
TensorCompare.cpp	CPU-strided-complex support for compa…	6 years ago
TensorConversions.cpp	Add memory_format support `to` and `ty`…	6 years ago
TensorFactories.cpp	CPU-strided-complex support for compa…	6 years ago
TensorFactories.h	Revert D15920763: Move TensorOption…	6 years ago

TensorIterator.cpp	Revert D18170997: Make TensorIterator…	6 years ago
TensorIterator.h	Revert D18170997: Make TensorIterator…	6 years ago
TensorIteratorReduce.cpp	Convert TensorIterator to use function_r…	6 years ago
TensorProperties.cpp	Port is_set_to from TH/THC to ATen.	6 years ago
TensorShape.cpp	provide memory format as Contiguous e…	6 years ago
TensorTransformations.cpp	Address review comments on #28011.	6 years ago
TensorTransformations.h	Replace AT_CHECK with TORCH_CH…	6 years ago
TriangularOps.cpp	Migrate implementations of triu and tril t…	6 years ago
TriangularOpsUtils.h	Migrate implementations of triu and tril t…	6 years ago
TypeProperties.cpp	Address review comments on #28011.	6 years ago
TypeProperties.h	Address review comments on #28011.	6 years ago
UnaryOps.cpp	Migrate `sinh` and `sinh_` from the TH t…	6 years ago
UnaryOps.h	Vectorized complex unary and binary op …	6 years ago
Unique.cpp	Enabled torch.unique for bool tensors (#…	6 years ago
UpSample.h	Replace AT_CHECK with TORCH_CH…	6 years ago
UpSampleBicubic2d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
UpSampleBilinear2d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
UpSampleLinear1d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
UpSampleNearest1d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
UpSampleNearest2d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
UpSampleNearest3d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
UpSampleTrilinear3d.cpp	Deprecate tensor.data<T>(), and codemo…	6 years ago
VariableMethodStubs.cpp	C++ API parity: at::Tensor::requires_grad_	6 years ago
WeightNorm.cpp	Replace AT_CHECK with TORCH_CH…	6 years ago
c10_utils.h	c10::KernelFunction (#26337)	6 years ago
im2col.h	Port im2col and vol2col (#21769)	6 years ago

im2col_shape_check.h	cleanup warnings	6 years ago
layer_norm.cpp	Eliminate some unnecessary tensor ref c…	6 years ago
layer_norm.h	Add fused layer norm impl on CUDA in …	6 years ago
native_functions.yaml	Migrate nll_loss from TH to ATen (CPU)…	6 years ago
vol2col.h	Port im2col and vol2col (#21769)	6 years ago

README.md

ATen "native" functions are the modern mechanism for adding operators and functions to ATen (they are "native" in contrast to legacy functions, which are bound via TH/THC cwrap metadata). Native functions are declared in `native_functions.yaml` and have implementations defined in one of the `cpp` files in this directory.

Like all ATen methods/functions, native functions are made available from both ATen's C++ and Python APIs. In C++, they are made available either as methods on `Tensor` (`t.mymeth()`) and functions in the ATen namespace (`at::myfunc()`). In PyTorch, they are made available as methods on `Variable` or as functions on `torch._C._FunctionBase` (it is the user's responsibility to re-exporting these functions in a more user-facing module.) At the moment, only functions which ingest `Variable` are made available; to use a function with non-differentiable tensors, wrap your tensors with `Variable` before passing them in.

The rest of this document describes how to implement an ATen function.

Registering a function in `native_functions.yaml`

Every native function must have an entry in `native_functions.yaml`. The format can be summarized as:

```
- func: func_name(ArgType arg0[=default], ArgType arg1[=default], ...)
  variants: function, method
  dispatch:
    CPU: func_cpu
    CUDA: func_cuda
```

Each component is described in more detail below:

`func`

```
- func: func_name[.overload_name](ArgType arg0[=default], ArgType arg1[
```

The `func` entry is a string describing the name of the function and its type signature.

Argument types. These types are permissible as ArgType:

- `Tensor`. A `Tensor` argument translates into a C++ argument of type `const Tensor&` (except when the argument is "inplace"; in this case, it is simply `Tensor&`). A trailing `?`, as in `Tensor?`, indicates that the tensor argument is optional and may be omitted by passing an undefined tensor. When a function takes multiple `Tensor` arguments, these tensors are assumed to be the same type (e.g., if one argument is a `FloatTensor`, all other arguments are checked to be `FloatTensor`s). `Tensor` or `Tensor?` must sometimes be annotated to indicate aliasing and mutability. In general annotations can be defined via the following four situations:
  - `Tensor(a)` - `a` is a set of Tensors that may alias to the same data.
  - `Tensor(a!)` - `a` members of a may be written to thus mutating the underlying data.
  - `Tensor!` - shorthand for Tensor(fresh_identifier!)
  - `Tensor(a! -> a|b)` - Tensor is in set `a`, written to, and after the write is in set `a` AND `b`. For more details on when and why this needs to happen, please see the section on annotations.
- `Tensor[]`. A `Tensor[]` argument translates into a C++ argument of type `ArrayRef<Tensor>` (a.k.a. `TensorList`)
- `int[]`. `int[]` accepts an optional length specifier, e.g., `int[2]`, which has no effect in C++ but extends our Python bindings to accept a bare number, which will be expanded into an appropriately sized list by repeating the number.
- `int`. Think about this like a Python int. This is translated into a C++ argument of type `int64_t`.
- `float`. Think about this like a Python `float`. It is translated into a C++ argument of type `double`.
- `bool`
- `str`
- `Scalar`. `Scalar` supports binding to any numerical types from Python, including integral types, floating point types, and zero dimensional tensors. `int` and `float` bind to the corresponding Python numerical types. However, you probably don't want to use `Scalar`. It's really used for binding to TH/THC code "real" types where the Python APIs you are binding to are actually different types. `float` and `int` argument types should suffice for most algorithms.
- `Generator?`, the state for a random number generator,
- `bool[N]` (where N is `1-4`).
- `TensorOptions`. Tensor options provide information about how a tensor should be constructed; it is most useful when you are writing a factory function, where you have no `Tensor` inputs and thus cannot otherwise determine how to construct a `Tensor`.

- `*` is a special sentinel argument, which doesn't translate into an actual argument, but indicates that in the Python bindings, any subsequent arguments must be specified as keyword arguments (and cannot be provided positionally).
- `?` is trailing question mark that annotates an argument to be an optional type. Grep for `optional` to find some example usages. In general, most functions will not need to use this, but there are some cases that we want to use optional for the different types:
  - You want to pass a `None` to an ATen function/method from Python and handle the None type on the C++ side. For example, `clamp(Tensor self, Scalar? min=None, Scalar? max=None)` can take `None` for its `min` and `max` parameter, but does not dispatch to different backends if one of the parameters is `None`. Optional type can accept a `None` type (`nullopt` in C++) from Python and use the C++ Optional class to interact with the parameters.
  - You want a default value, which is fine in Python, but would cause ambiguity in C++. For example, `norm(Tensor self, Scalar p=2, int dim, bool keepdim=False)` would cause ambiguity in C++ since its default args must be adjacent (`p` could not have a default value when `dim` does not). Therefore, we need to make `p` as a optional Scalar, and make `p=2` when `p` is not passed in (nullopt).
  - You want a value to default to the same value as another argument (this cannot be expressed in C++ default arguments).

Functions with no tensor inputs are called *factory functions*, and are handled specially by code generation. If your function is behaving differently than another example, check first and see if one is a factory while another is not. In some rare cases, factory function might have a tensor argument. In this case mark it with 'category_override: factory' explicitly.

Argument names. Argument names are meaningful; downstream binding code may make use of the specific argument name you provide, and a rename of an argument name is considered a BC-breaking change (e.g., you will probably need to update `tools/autograd/derivatives.yaml` at least). For more details please see the section on `variants`.

As a convention we use 'out' to indicate an output argument. This aligns with the Python bindings. Even if a function might not be used in the Python bindings, we still advise to follow this convention. Check the generated code when making a change to make sure you're not breaking the API when renaming an argument name of an existing function.

TODO: Do argument names affect Python keyword arguments?

Defaults. Any suffix of arguments can have a default value defined; these default values translate into C++/Python default values which are applied when those positional arguments are not specified.

Here are the supported default values:

- Numbers (e.g., `0` or `5.0` for `int`, `float` and `int[]` with an explicit length (e.g., `int[2]`)--in the case of `int[]` a number is replicated to fill the length (e.g., `int[2]`

`x=2` is equivalent to `int[2] x=[2,2]`).

- Lists of numbers (e.g., `[0, 0]`) for `IntList`.
- Booleans (e.g., `True`) for `bool`.
- Empty initializer lists (e.g., `[]`) for `Tensor` (this implicitly changes a `Tensor` argument to accept undefined tensors).
- `None` for pointer types (e.g., `Generator?`)

Returns. The following are permissible on Return:

Non-tuple return:

```
ReturnType [retarg0]
```

Tuple return:

```
(ReturnType [retarg0], ReturnType [retarg1], ...)
```

The following are permissible on ReturnType:

- `Tensor` and `Tensor[]`, which translate into the C++ types `Tensor` and `std::vector<Tensor>`, respectively (unless the operation is in-place, in which case the return type is `Tensor&`.
- A tuple of any number of `Tensor`, e.g., `(Tensor, Tensor)`, translating into the C++ `std::tuple<Tensor, Tensor>`.

If you need a type that is not listed in this list, it may be possible to extend ATen's code generation to support it. ATen's philosophy on types to support is that it supports only simple, universal types, as well as a handful of fundamental Tensor structures (e.g., `Tensor` and `Generator?`), because these types can be easily ported to any language bound to ATen (in practice, C++ and Python.)

Return also supports specifying (optional) return argument names. These serve two functions:

- They let you easily write derivatives in terms of return arguments in `tools/autograd/derivatives.yaml`
- They correspond to the named field the output can be referred to from Python. (This means that changing a return argument name is BC-breaking, be careful!)

Note that argument type modifiers such as defaults and optional are not currently supported on Return.

Overloads. You can register multiple functions with the same name and different function signatures if you give them unique overload names. An overload name is specified after the function name, separated by a dot.

Overload names do not have to be globally unique, but must be unique in the set of all overloads for the same function. Overload names cannot be changed for backwards compatibility reasons. Please try to make overload names semantically meaningful. An overload name that just enumerates all the argument types isn't helpful. In many cases, a semantic name is clear from what the overload is doing differently. As a fallback, you can use the name or type of the first differing argument as an overload name.

If you add a new overload to an existing function, please leave the existing overload names as they are (for backwards compatibility), but give the new overload a new, unique name.

Not specifying an overload name is equivalent to specifying an empty overload name. If you add a new function with multiple overloads, give them unique overload names, at most one overload is allowed to have an empty overload name.

The declarations also support the following attributes.

`variants`

```
variants: function, method
```

Controls whether Tensor method (`t.foo()`) or namespace Function (`at::foo()`) is generated as a result of this declaration. If the declaration is a method, you must have an argument `Tensor self` at some position in the method; in the method variant this argument will be elided from the argument list. For example, given the declaration `where(BoolTensor cond, Tensor self, Tensor other)`, this generates the function `at::where(cond, self, other)` and the method `self.where(cond, other)`.

By default, ATen generates only the function variant for a native function. When should you also generate a method variant? Tensor operations as methods are appropriate for "core" Tensor operations (e.g., add, sub, etc.), but not for more complicated neural network layers (e.g., `conv2d`) and internal functions designed specifically for binding (e.g., `cudnn_convolution`).

As we progress along our schema unification of the `func` schema with the JIT signature schema, we must introduce features that allow us to increase compliance. One of these features are Tensor annotations. As of now we use naming conventions to indicate whether an argument of a function is going to be mutated and returned.

`annotations`

There are two typical situations in which we mutate the memory of an argument in the Python frontend: a) For an inplace operations such as `self.abs_()` b) for a function with an output keyword argument such as `torch.abs(input, out=None)`.

In order to provide implementations for these Python functions the legacy schema requires C++ implementations for three situations `abs(Tensor self) -> Tensor`, `abs_(Tensor`

`self) -> Tensor` and `abs_out(Tensor out, Tensor self) -> Tensor`.

Now, as we move towards the unification, we start to use a different syntax to represent this by using annotations. In the end we still translate to the legacy schema for the downstream consumers such as the C++ code generation, but this will soon change.

If two Tensors carry the same annotation, they both *may* represent the same memory. A write annotation, as indicated by an exclamation mark, indicates that they both *may* also be written to.

Let's revisit the previous native function declarations and see the conventions of adding annotations.

- `abs(Tensor self) -> Tensor` stays the same as it will always allocate new memory.
- `abs_(Tensor(a!) self) -> Tensor(a!)` `self` may be written to and returned. Further, the annotation indicates that the return value may alias the input. This indicates an inplace function and by convention ends in a single '_'.
- `abs(Tensor self, *, Tensor(a!) out) -> Tensor(a!)` In the Python frontend `out` can be passed as a keyword argument and may be written to. In this case it indicates the schema for a function that must accept `out` as this does not provide a default argument. The idea behind representing this as a optional argument is to document the intended usage. This maps to the legacy `abs_out(Tensor out, Tensor self) -> Tensor`. As with the legacy `_out` function you must call the argument `Tensor out` or `Tensor out0`, `Tensor out1` in the context of multiple arguments.

There is also another situation in which we use annotations, namely views.

- `transpose(Tensor(a) self, int dim0, int dim1) -> Tensor(a)` An alias to the memory represented by `self` may be also returned, however it is not mutated.

We have some asserts to check whether a developer uses these annotations correctly and throw asserts if she doesn't. For example, any out function must use the `(a!)` annotation as described above. If this causes a lot of confusion please add @cpuhrsch to your PR.

`dispatch`

```
dispatch:
    CPU: func_cpu
    CUDA: func_cuda
```

This specifies the actual name of the function you want to dispatch to, so you can dispatch to different functions depending on whether or not you have CPU or CUDA tensors. Technically, it is also possible to write `dispatch: func_name` to unconditionally dispatch to a native function whose name is different than the name in the public ATen API, but this is generally frowned upon (just name them the same thing!)

`device_guard`

```
device_guard: False
```

By default, ATen code generation will generate a DeviceGuard invocation, which will ensure that kernel code will run with the current device set to match the device of the first Tensor argument (or first tensor of the first Tensor[] argument, if the function takes a list of tensors). For the most part, this means kernel authors do not have to worry about setting devices.

However, in some cases, setting the device is unnecessary, because, e.g., you call a function already manages device guard setting, or you're a function that simply does not interact with any devices. In that case, code generation of the device guard can be disabled by adding `device_guard: False` to your function definition.

Note. We are considering eliminating automatic generation of DeviceGuard, in which case this field would go away. If you have an opinion on the matter, please write in at #14234

`supports_named_tensor`

```
supports_named_tensor: True
```

Experimental: this option is ignored unless compiling with BUILD_NAMEDTENSOR=1. By default, (`supports_named_tensor: True`) ATen code generation will generate a check that all tensor inputs to the function are unnamed. This is used to incrementally implement named tensors; if a function supports named tensors, then it'll have `supports_named_tensor: True`; otherwise, passing it a named tensor will error out.

`matches_jit_signature`

```
matches_jit_signature: False
```

This will indicate that the func syntax does not follow the JIT signature schema. If you are a triggering an assert related to JIT signature compliance try adding this field and setting it to False. In general, this serves as a means of tracking an ongoing schema unification with the goal of aligning func syntax with other components of PyTorch in order to reduce overall complexity. If you find yourself having to set this field to False add @gchanan to your PR's set of reviewers.

`use_c10_dispatcher`

```
use_c10_dispatcher: 'no'
use_c10_dispatcher: 'unboxed_only'
use_c10_dispatcher: 'full'
```

This will indicate that the func signature only uses features supported by the c10 dispatcher. With this flag, the operator will be added to the c10 operator library and be available there. If setting this to 'full' works for your operator, please do. For a few corner cases, enabling this might not compile successfully, so setting this to 'unboxed_only', or as last resort 'no' is a workaround. Also, 'no' is the default if you don't specify anything.

Writing an implementation in C++

Implementations of native functions go in an appropriate C++ file in the `native/` directory (they are organized roughly by topic, but there is no semantic meaning to their organization aside for the `cuda` directory, which is the only place the build system knows how to build `cu` files.) To write a native function, you only need to write a C++ implementation (no header necessary) with a matching signature to the generated header from the ATen metadata There are many simple native functions; take a look at some of them to see what to do.

Although writing an ATen function is mostly writing the algorithm you want to implement, there are some less obvious details you should also consider.

Will your function be automatically differentiable?

If you are writing a pair of functions `foo` and `foo_backward`, with the intent that `foo_backward` implements the derivative of `foo`, then your implementation of `foo` is probably not automatically differentiable: it might make use of functions like `data_ptr()` or it dispatches differently depending on if it's operating on CPU or CUDA tensors. Once you write these two functions, you will have to write an entry correlating them together in `tools/autograd/derivatives.yaml`.

However, in some situations, you can write a function in ATen and it will be automatically differentiated! This can be the case if the function implementation only calls other operations which are themselves differentiable. In this case, you don't have to write an entry in `tools/autograd/derivatives.yaml`.

Will this function be exposed to python? What are the namespaces?

We don't generate python bindings for all functions. There're certain patterns in function name that we skip in python binding generation, e.g. `*_backward`. Check `tools/autograd/gen_python_functions.py` for the latest rules.

The generated bindings are either exposed as methods on python_variable or functions on torch._C._nn object(marked with `python_module: nn`).

Can it handle being passed Variables?

The biggest subtlety of writing an ATen implementation is the fact that `Tensor` is not a "final" class: your implementation may be passed objects which inherit from `Tensor` (in

particular, the `Variable` subclass implements automatic differentiation in PyTorch.) This has some direct consequences on valid implementations:

- Never create a `Tensor` directly (e.g., `at::CPU` or `at::CUDA`), as a caller will be expecting to get `Variable`s out if it passes `Variable`. Instead, create tensors using the `options()` of one of the input tensors. E.g., `at::empty(sizes, input.options())` or `at::ones(input.options().dtype(kByte))`, if you need a different scalar type.
- If you need to call other ATen functions, be sure to qualify the call with `at::`; don't call them unqualified (in the `at::native` namespace). Using the qualified name ensures that your invocation gets dispatched to the `Variable` (which may be overridden to behave differently than simply dispatch to `at::native`).

These are not hard and fast rules: in particular, if you explicitly define a derivative for a function, it will only ever be called with `Tensor` arguments. However, it is considered good style to abide by these rules, since code written in this style is more robust.

NB: There is one downside to following the `at::` qualification rule, which is that if you know that you will only ever be called with `Tensor`, a direct `at::native` call will be more efficient (as it avoids a dynamic dispatch).

How to handle broadcasting?

Unlike our legacy TH bindings, ATen native functions do not automatically handle broadcasting; you will have to insert the necessary broadcasting calls yourself.

When writing broadcasting code, we obey the convention that `op` is broadcasting, while `s_op` (with the `s_` prefix) is not broadcasting. The relationship is best seen by an example of how you would implement broadcasting addition out of non-broadcasting addition:

```
#include <ATen/ExpandUtils.h>

Tensor add(const Tensor& self, const Tensor& other) {
  Tensor b_self, b_other;
  std::tie(b_self, b_other) = expand_outplace(self, other, "add");
  return s_add(b_self, b_other);
}

Tensor s_add(const Tensor& self, const Tensor& other) {
  // non-broadcasting implementation of addition
}
```

For inplace operations, the convention looks like this:

```
Tensor& add_(Tensor& self, const Tensor& other) {
  Tensor b_other = expand_inplace(self, other, "add_");
  return s_add_(self, b_other);
}
```

```
Tensor& s_add_(Tensor& self, const Tensor& other) {
  // non-broadcasting implementation of inplace addition
}
```

Undefined tensor conventions

By default, `Tensor` arguments to ATen functions are always defined, unless you explicitly specified that an undefined tensor was permissible by writing `Tensor?` or `Tensor? x=[]`, the latter one is needed when you have to assign a default value in C++ (e.g. in the middle of other parameters with default values).

The rules for returning undefined Tensors are a bit more subtle, but there is only one case you have to remember:

- If the function in question is a backward function which accepts a `std::array<bool,N> output_mask` argument, you MUST return an undefined `Tensor` at every tuple position `i` for which `output_mask[i]` is false, otherwise
- You MUST NOT return an undefined tensor.

The most common situations where you might be tempted to return undefined tensors are when:

- You have a forward function that may return a buffer if training is enabled, but does not return the buffer in inference mode. In this case, just return an appropriately typed zero-size tensor.
- You have a backward function where the gradient for an input is zero. In this case, you are expected to create a zero-filled tensor of appropriate size to return for this input. To get the shape, it may be helpful to take a `TensorGeometry` of the input to use.

Debugging tips

If you build ATen and get a linker error, that probably means you copy-pasted the C++ definition of your function incorrectly. Double check your `Tensor` arguments, and make sure you wrote `const Tensor&` in your signature.